

Mind the Interface: Repairing Speculative Drafters for Post-Trained Language Models

Anonymous ACL submission

Abstract

Speculative decoding with target-conditioned drafters such as EAGLE-3 is among the fastest ways to serve large language models, and the drafter released with a base model is routinely reused for its many fine-tuned derivatives. We show that this reuse breaks for an important and growing class of models: derivatives produced by distilling a stronger reasoning teacher. In a census of 174 public derivatives, quantized, merged and RL-tuned models retain 98–100% of the family drafter’s acceptance, while DeepSeek-R1-Distill-Llama-8B and Llama-3.1-Nemotron-Nano-8B lose up to 30%, erasing most of the speedup. We trace this loss to the drafter’s *feature interface*, the projection through which it reads the target’s hidden states, and propose *interface repair*: re-fitting the family drafter on the derivative’s own responses to public instructions, without any of the original training data. Re-fitting the interface alone (12% of the drafter’s parameters) closes 60% of the gap to a dedicated drafter trained specifically for the derivative, and a full warm start closes 72%, using 11 GPU-hours, over $400\times$ less compute than the public recipe for a dedicated drafter. On SPEED-Bench and MATH-500, repair raises the end-to-end speedup of R1-Distill from $1.31\times$ to $1.83\times$ (dedicated drafter: $2.05\times$) and of Nemotron-Nano from $1.29\times$ to $1.80\times$, outperforming independent draft models and training-free drafting.

1 Introduction

Speculative decoding has become the standard way to serve large language models (LLMs) quickly: a small drafter proposes several tokens, and the target model verifies all of them in a single forward pass, accelerating generation without changing its output (Leviathan et al., 2023; Chen et al., 2023). The fastest drafters today are *target-conditioned*. Rather than modeling text from scratch, they read the target’s own hidden states and predict its next

tokens from them (Cai et al., 2024; Li et al., 2024b, 2025; Chen et al., 2026b). EAGLE-3, for instance, fuses features from the target’s lower, middle and upper layers and reports speedups of up to $6.5\times$ (Li et al., 2025). This tight coupling is also their price: each drafter is trained for one specific target, on hundreds of thousands of examples, for tens of epochs.

Open-weight models, however, rarely stay single models. A popular base model such as Llama-3.1-8B-Instruct spawns thousands of derivatives on public model hubs, which are quantized, merged, aligned, RL-tuned and distilled (Horwitz et al., 2025). Training a dedicated drafter for each derivative is out of reach, so serving stacks reuse the drafter released for the parent, the *family drafter*, for all of its descendants (Kwon et al., 2023; Red Hat AI and vLLM Project, 2025). Practitioners make this choice every day, yet two questions behind it remain open: when does a family drafter keep working for a derivative, and what can be done when it does not?

We answer the first question with a census of 174 public derivatives in the Llama-3.1-8B and Qwen3-8B families (Figure 1a). For most of the ecosystem, reuse is reliable. Quantized, merged, ablated and on-policy RL derivatives retain 98–100% of the family drafter’s acceptance, in line with evidence that on-policy training keeps models close to their initialization (Shenfeld et al., 2026; Chen et al., 2026a). Reuse fails, however, for precisely the derivatives that are most expensive to serve: models distilled from stronger reasoning teachers. DeepSeek-R1-Distill-Llama-8B and Llama-3.1-Nemotron-Nano-8B lose 26–30% of first-token acceptance under both EAGLE-3 and DFlash drafters. For R1-Distill, the speedup collapses from $2.05\times$ with a dedicated drafter to $1.31\times$ (Figure 1b). Reasoning models routinely emit thousands of tokens per query, so this lost speedup costs the most exactly where acceleration

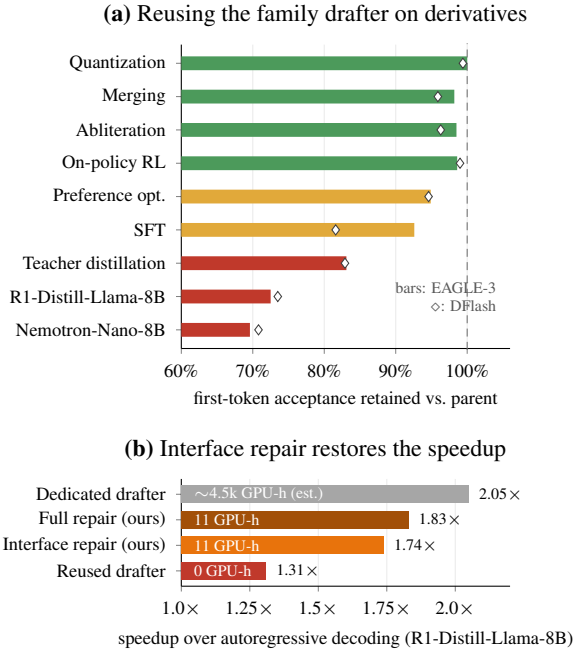


Figure 1: **(a)** First-token acceptance of the Llama-3.1-8B-Instruct family drafters on public derivatives, relative to the parent model (174 derivatives grouped by post-training history; EAGLE-3 bars, DFlash diamonds). Weight-space edits and on-policy RL leave the drafter intact; distillation from a reasoning teacher breaks it. **(b)** On R1-Distill-Llama-8B, interface repair recovers most of the speedup of a dedicated drafter at a small fraction of its training cost.

matters most.

We then show that the failure is localized. A target-conditioned drafter accesses its target through a single learned map, the *feature interface*, a dense projection that compresses the target’s multi-layer hidden states into the drafter’s input space. Post-training shifts these hidden states (Kumar et al., 2022), so the interface misreads them. The rest of the drafter, namely its decoder layer and language-modeling head, encodes knowledge about drafting that transfers largely intact. This observation suggests *interface repair* (Figure 2): elicit the derivative’s own responses to public instructions, then re-fit the family drafter on them with its native training objective, updating either the interface alone or the whole warm-started drafter. Repair needs no access to the derivative’s or the drafter’s training data, no new architecture, and no change to the serving engine.

Interface repair is highly effective. On R1-Distill-Llama-8B, re-fitting only the interface closes 60% of the gap between the reused fam-

ily drafter and a dedicated drafter trained by the EAGLE authors, and a full warm start closes 72%. With 11 GPU-hours of compute, over $400\times$ less than the public recipe for a dedicated drafter, repair lifts the end-to-end speedup from $1.31\times$ to $1.83\times$. On Nemotron-Nano, for which no dedicated drafter exists, it lifts the speedup from $1.29\times$ to $1.80\times$. Repaired drafters are faster than training-free n-gram drafting and independent small draft models, and the gains hold across drafter checkpoints and drafter families. In summary, we make three contributions:

- We present a census of family-drafter reuse across 174 public derivatives and identify off-policy distillation as the post-training regime that breaks target-conditioned drafters (§3).
- We localize the failure to the drafter’s feature interface and propose interface repair, a simple recipe that re-fits the family drafter on the derivative’s own outputs without any original training data (§4).
- We show on SPEED-Bench and MATH-500 that repair recovers most of a dedicated drafter’s speedup at a small fraction of its cost, across targets, drafter checkpoints and drafter families (§6).

2 Background: Target-Conditioned Drafters

Speculative decoding. Given a prefix $y_{\leq t}$, a drafter proposes K tokens $\hat{y}_{t+1:t+K}$. The target scores all of them in one forward pass and accepts the longest prefix that agrees with its own predictions, plus one bonus token that it generates itself (Leviathan et al., 2023; Chen et al., 2023). The output is identical to decoding with the target alone. Efficiency is governed by the acceptance length τ , the expected number of tokens produced per target forward pass, and by the cost of drafting. With per-step costs c_T for the target and c_D for the drafter, the speedup is approximately $\tau c_T / (c_T + K c_D)$. A good drafter is therefore both *accurate* (high τ) and *cheap* (low c_D).

Target-conditioned drafting. Target-conditioned drafters achieve both properties by reusing the target’s computation. Medusa attaches extra decoding heads to the target’s final layer (Cai et al., 2024), and EAGLE drafts autoregressively at the feature level (Li et al., 2024b,a). EAGLE-3 (Li et al., 2025) reads hidden

states $h_t^{\text{low}}, h_t^{\text{mid}}, h_t^{\text{high}} \in \mathbb{R}^d$ from three depths of the target and maps them through a learned *feature interface* $W_{\text{fc}} \in \mathbb{R}^{d \times 3d}$.

$$g_t = W_{\text{fc}} [h_t^{\text{low}}; h_t^{\text{mid}}; h_t^{\text{high}}]. \quad (1)$$

The fused feature g_t is combined with the embedding of the next token and passed through a single decoder layer and a language-modeling head over a reduced draft vocabulary, producing the draft distribution q_θ . EAGLE-3 is trained with *training-time test*: the drafter is unrolled on its own predictions for $k = 1, \dots, k_{\text{max}}$ steps and matched to the target’s next-token distribution p_T at each step,

$$\mathcal{L}(\theta) = \sum_{k=1}^{k_{\text{max}}} \mathbb{E}_t \text{KL} \left(p_T(\cdot \mid y_{\leq t+k}) \parallel q_\theta^{(k)} \right), \quad (2)$$

so that it learns to draft from its own predictions as it does at inference. DFlash (Chen et al., 2026b) drafts an entire block in parallel with a lightweight diffusion model, but it reads the target through the same kind of multi-layer feature projection.

Family drafters. A drafter is trained once for a base model and released publicly. Serving engines then load it for any model that shares that base model’s architecture and tokenizer (Kwon et al., 2023; Red Hat AI and vLLM Project, 2025). We call the base model the *parent* T , its drafter the *family drafter* D_{θ_0} , and the model being served the *derivative* T' .

3 When Family Drafters Fail

A census of derivatives. We collect 174 public derivatives in the Llama-3.1-8B and Qwen3-8B families (Grattafiori et al., 2024; Yang et al., 2025) from the Hugging Face hub, all of which load with the architecture and tokenizer of the instruction-tuned parent and its released drafters. We label each by its post-training history as described in its model card: quantization, merging, ablation, supervised fine-tuning (SFT), preference optimization, on-policy reinforcement learning, or distillation from a teacher. For every derivative, we serve the family EAGLE-3 and DFlash drafters with vLLM (Kwon et al., 2023) and measure *first-token acceptance*, the probability that the target accepts the drafter’s first proposed token, on identical prompts for the derivative and its parent. We report retention, the derivative’s acceptance divided by the parent’s. First-token acceptance is independent of response length, which itself changes with

post-training, and so isolates how well the drafter predicts the derivative.

Distillation breaks the drafter. Figure 1a shows a clear ordering. Edits in weight space leave the drafter intact: quantized derivatives retain 100% of the parent’s acceptance, merged ones 98% and ablated ones 99%. On-policy RL does not disturb it either (99%). Off-policy supervision erodes acceptance progressively, from 95% for preference optimization and 93% for SFT to 83% for distillation from a teacher. The most severe losses occur for the two widely used reasoning distills. R1-Distill-Llama-8B, trained on the outputs of DeepSeek-R1 (DeepSeek-AI, 2025), retains 72.5% under EAGLE-3 and 73.5% under DFlash. Nemotron-Nano-8B (Bercovich et al., 2025) retains 69.6% and 70.8%. This ordering mirrors how far each regime moves a model from its initialization. On-policy RL learns from the model’s own samples and stays close to it (Shenfeld et al., 2026; Chen et al., 2026a), whereas distillation fits the model to a different model’s distribution (Hinton et al., 2015; Kim and Rush, 2016) and reshapes its internal representations.

The cost of this loss is substantial. On R1-Distill, the reused drafter reaches an acceptance length of only 1.76 tokens per step on SPEED-Bench, compared with 2.85 for a dedicated EAGLE-3 drafter that the EAGLE authors trained on R1-Distill. In end-to-end decoding, this is the difference between $1.31 \times$ and $2.05 \times$ (§6). A dedicated drafter is rarely an option: the public EAGLE-3 recipe trains on more than half a million examples for tens of epochs, and for most derivatives, including Nemotron-Nano, no dedicated drafter exists at all. Practitioners can, however, tell cheaply when reuse is at risk. Measuring acceptance on just 16 requests identifies the degraded derivatives in our census with an AUROC of 0.91 for EAGLE-3 and 0.93 for DFlash.

The failure lies in the interface. To find which part of the drafter breaks, we re-fit individual components of the R1-Distill family drafter on 256 of the derivative’s responses (§4) and measure how much of the gap to the dedicated drafter each closes (Table 1). Adapting the decoder layer with LoRA (Hu et al., 2022) closes only 10% of the gap, and recalibrating the statistics of the target’s features closes none. Re-fitting the dense feature interface closes 36%, nearly as much as updating the interface and decoder together (37%) and most of

Re-fitted component	Params	Gap closed
none (reuse)	0	0%
feature statistics (affine)	25K	0%
decoder layer (LoRA)	1.2M	10%
interface, rank 8	0.13M	5%
interface, rank 32	0.52M	15%
interface, dense	50M	36%
interface + decoder (LoRA)	52M	37%
entire drafter	425M	46%

Table 1: Which component should be re-fitted? Fraction of the acceptance-length gap between the reused family drafter and a dedicated drafter that each re-fit closes on R1-Distill-Llama-8B (SPEED-Bench, 256 repair examples). The dense feature interface alone accounts for most of the recoverable gain.

what fine-tuning the entire drafter achieves (46%). Low-rank updates of the interface fall well short (5–15%). This pattern indicates that post-training does not perturb the derivative’s features slightly but requires a full re-mapping of how they should be read. The drafter’s decoder and head remain good at drafting. What fails is the interface, which no longer understands the features of the model it serves.

4 Interface Repair

Given a derivative $\mathcal{T}'$ and the family drafter D_{θ_0} of its parent, interface repair produces a drafter D_{θ} that is matched to $\mathcal{T}'$. It does so without the data used to train either model. The recipe has three steps (Figure 2).

Eliciting the derivative. The drafter must predict what $\mathcal{T}'$ will generate, so we let $\mathcal{T}'$ show it. We sample instructions x from a public instruction corpus. We use 16k instructions from Alpaca (Taori et al., 2023), deduplicated against all evaluation prompts. We render each instruction with the derivative’s chat template and greedily decode a response $y' \sim \mathcal{T}'(\cdot | x)$ of up to 512 tokens. For a reasoning distill, these responses are long reasoning traces, exactly the text the drafter will face in deployment. This mirrors sequence-level distillation (Kim and Rush, 2016) and drafter alignment (Zhou et al., 2024; Goel et al., 2024), with one difference: only the drafter learns, and only to follow a fixed target.

Re-fitting the drafter. For each pair (x, y') , a single teacher-forced pass of $\mathcal{T}'$ provides, at every response position, the three hidden states that enter Eq. (1) and the next-token distribution $p_{\mathcal{T}'}$.

We minimize the drafter’s native training-time-test objective (Eq. 2) with $p_{\mathcal{T}'}$ in place of p_T and $k_{\max} = 3$, starting from the family drafter’s parameters θ_0 . We consider two operating points:

- **Interface repair** updates only W_{fc} , 50M parameters or 12% of the drafter, and keeps the decoder layer and head frozen.
- **Full repair** warm-starts all drafter parameters from θ_0 ; the token embedding remains shared with $\mathcal{T}'$.

Both variants inherit everything the family drafter already knows about drafting and only teach it to read the new target. This warm start is essential. The same architecture trained from scratch on the same data and budget falls below even the reused drafter (§6).

Cost and deployment. Repair takes one pass over 16k examples, or 7.9M response tokens. Generating the responses and training together take about 11 A40 GPU-hours. The repaired drafter has the same architecture, vocabulary and file format as the family drafter, so it is a drop-in replacement in the serving engine.

5 Experimental Setup

Targets and drafters. We study the two reasoning distills from the census: DeepSeek-R1-Distill-Llama-8B, distilled from DeepSeek-R1 into Llama-3.1-8B (DeepSeek-AI, 2025), and Llama-3.1-Nemotron-Nano-8B-v1, derived from Llama-3.1-8B-Instruct through distillation and reasoning post-training (Bercovich et al., 2025). Both share the Llama-3.1 architecture and tokenizer, and hence the family drafters of Llama-3.1-8B-Instruct. Our main family drafter is the EAGLE-3 drafter that the EAGLE authors released for Llama-3.1-8B-Instruct (Li et al., 2025). We also repair the production EAGLE-3 drafter distributed with vLLM’s speculators library (Red Hat AI and vLLM Project, 2025) and the DFlash drafter (Chen et al., 2026b). As an upper reference, we use the dedicated EAGLE-3 drafter that the EAGLE authors trained specifically for R1-Distill-Llama-8B.

Baselines. We compare against (i) reusing the family drafter unchanged; (ii) n-gram prompt lookup (Saxena, 2023), a training-free drafter built into vLLM; (iii) an independent draft model, Llama-3.2-1B-Instruct (Grattafiori et al., 2024),

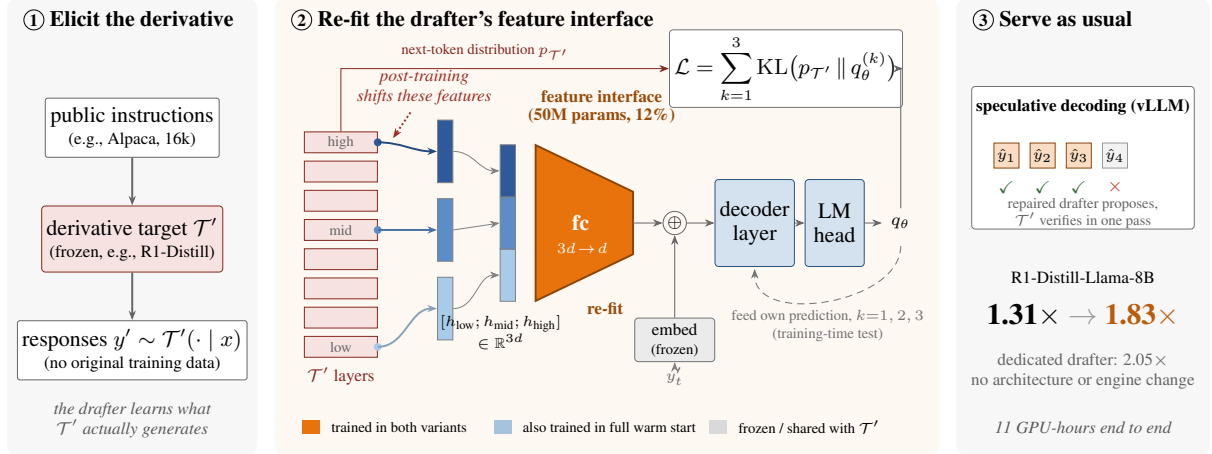


Figure 2: **Interface repair.** ① The derivative target $\mathcal{T}'$ answers public instructions, producing data that reflect what it actually generates; none of its training data, nor the drafter’s, is needed. ② A teacher-forced pass of $\mathcal{T}'$ exposes the multi-layer hidden states that the drafter reads and the next-token distribution it must match. Post-training has shifted these features, so we re-fit the drafter’s feature interface (orange) with its native training-time-test objective, optionally warm-starting the decoder layer and head as well (blue). ③ The repaired drafter keeps its architecture and format, so it is served with an unchanged engine.

which shares the targets’ tokenizer; and (iv) training the same EAGLE-3 architecture from scratch on our repair data with the same budget. All speculative methods propose $K=4$ tokens per step.

Benchmarks and metrics. We evaluate on SPEED-Bench (Abramovich et al., 2026), a unified benchmark for speculative decoding, using a stratified 128-prompt sample of its qualitative split, which spans eleven domains from coding and math to writing, roleplay and multilingual chat. For long-form reasoning, we use MATH-500 (Hendrycks et al., 2021; Lightman et al., 2024). Prompts are rendered with each target’s chat template, and responses are generated greedily for up to 512 tokens. We report the acceptance length τ (including the bonus token) and, for R1-Distill, the *gap closed*, $(\tau_{\text{repair}} - \tau_{\text{reuse}}) / (\tau_{\text{dedicated}} - \tau_{\text{reuse}})$. We also measure end-to-end wall-clock speedup over autoregressive decoding in vLLM on NVIDIA A40 GPUs at batch sizes 1 and 8. Main R1-Distill results on SPEED-Bench average three training seeds, whose gap closed varies by at most 2.5 points. Further details are in Appendix A.

6 Results

6.1 Main Results

Table 2 summarizes our results. On R1-Distill-Llama-8B, full repair raises the acceptance length from 1.76 to 2.54 on SPEED-Bench and from 1.93 to 2.97 on MATH-500, closing 72% and 54% of the

gap to the dedicated drafter. Interface repair alone closes 60% and 44% while updating only 12% of the parameters. These gains carry over to wall-clock time. At batch size 1, the repaired drafters accelerate decoding by 1.83 $\times$ and 1.74 $\times$, against 1.31 $\times$ for the reused drafter and 2.05 $\times$ for the dedicated one. In other words, full repair delivers 79% of the dedicated drafter’s acceleration over autoregressive decoding for an estimated 0.25% of its training compute. On Nemotron-Nano, where no dedicated drafter exists and repair is the only way to obtain a matched drafter, the speedup rises from 1.29 $\times$ to 1.80 $\times$. The improvements persist under batching, where speculative decoding has less idle compute to exploit: at batch size 8, repair lifts the speedup from 1.07 $\times$ to 1.29 $\times$ on R1-Distill and from 1.13 $\times$ to 1.42 $\times$ on Nemotron-Nano.

The family drafter is the asset. Training the same EAGLE-3 architecture from scratch on the repair data reaches an acceptance length of only 1.52 on SPEED-Bench, below even the unrepaired family drafter. A drafter’s ability to draft is expensive to acquire and is exactly what the family drafter already has. Repair preserves this ability and spends its budget on the one component that is out of date.

Repair versus other drafters. Training-free n-gram lookup rarely finds long matches in open-ended generation and stays below 1.6 tokens per step. The independent Llama-3.2-1B drafter pre-

Method	Training GPU-h	R1-Distill-Llama-8B				Nemotron-Nano-8B			
		$\tau \uparrow$		Speedup $\uparrow$		$\tau \uparrow$		Speedup $\uparrow$	
		SPEED	MATH	bs 1	bs 8	SPEED	MATH	bs 1	bs 8
Autoregressive decoding	–	1.00	1.00	1.00×	1.00×	1.00	1.00	1.00×	1.00×
N-gram prompt lookup	–	1.49	1.59	–	–	1.56	1.49	–	–
Independent drafter (Llama-3.2-1B)	–	2.44	2.97	1.26×	1.11×	2.65	2.84	1.37×	1.24×
Family drafter, reused	0	1.76	1.93	1.31×	1.07×	1.76	1.70	1.29×	1.13×
Family architecture, from scratch	11	1.52	1.67	–	–	–	–	–	–
Interface repair (ours)	11	2.41	2.77	1.74×	1.27×	2.42	2.72	1.69×	1.39×
Full repair (ours)	11	2.54	2.97	1.83×	1.29×	2.49	2.91	1.80×	1.42×
Dedicated drafter (reference)	~4.5k	2.85	3.84	2.05×	1.41×	<i>not available</i>			

Table 2: **Main results.** Acceptance length τ on SPEED-Bench and MATH-500 and end-to-end speedup over autoregressive decoding (vLLM, A40, batch sizes 1 and 8). Repairing the family drafter recovers most of a dedicated drafter’s speedup and is the fastest method that does not require a per-derivative drafter trained from scratch. Training cost includes generating the repair data; the dedicated drafter’s cost is estimated for the public EAGLE-3 recipe.

dicts the derivatives well, since it is itself a capable language model, but it must run sixteen transformer layers for every drafted token. Its drafting cost consumes most of its acceptance advantage, and it reaches only 1.26× on R1-Distill and 1.37× on Nemotron-Nano. Repaired drafters combine high acceptance with the near-free drafting of a single-layer, target-conditioned head and are therefore the fastest option on both targets and at both batch sizes.

6.2 Scaling and Cost

Figure 3a shows that repair improves steadily with data. With the production drafter, the gap closed by full repair grows from 46% with 0.5k examples to 58% with 4k, 66% with 16k and 71% with 64k. Every increase in data helps, and the curve is still rising at 64k examples. The EAGLE authors’ family drafter starts from a better position and stays ahead at every budget (64% at 4k, 72% at 16k). Interface repair follows a parallel curve 12–15 points lower and thus captures most of the benefit of the full warm start with an eighth of the trainable parameters.

Figure 3b places these results in terms of compute. Even 0.45 GPU-hours of repair on 256 examples already yield a 1.67× speedup, and 11 GPU-hours reach 1.83×. By contrast, we estimate that reproducing the dedicated drafter with the public EAGLE-3 recipe requires between 1.3k and 19k A40 GPU-hours, depending on the number of epochs. Repair thus recovers most of the dedicated drafter’s speedup for two to three orders of magnitude less compute. Because repair only needs the

Family drafter	Target	Reuse	Interface	Full
EAGLE-3 (authors)	R1	1.76	2.41	2.54
	Nemotron	1.76	2.42	2.49
EAGLE-3 (production)	R1	1.73	2.30	2.47
	Nemotron	1.82	2.32	2.42
DFlash (256 ex.)	R1	2.08	2.22	2.31
	Nemotron	2.08	2.24	2.36

Table 3: Interface repair generalizes across drafter checkpoints and drafter families. Acceptance length on SPEED-Bench; EAGLE-3 drafters use 16k repair examples, and DFlash drafts blocks of ten tokens and is repaired with only 256 examples.

derivative itself and public instructions, it can be applied the day a derivative is released.

6.3 Generality

Table 3 shows that the benefits of repair do not depend on a particular drafter. Repairing the production EAGLE-3 drafter distributed with vLLM yields the same picture as repairing the drafter of the EAGLE authors. Full repair closes 66% of the R1-Distill gap, interface repair closes 51%, and the end-to-end speedups are 1.81× and 1.72×. The method also transfers to a different drafter family. DFlash replaces autoregressive drafting with parallel block diffusion but reads the target through the same kind of feature interface, and repairing it with only 256 examples raises its acceptance length by 11–13% on both targets. Across all six settings, the pattern of Table 1 holds: the interface carries most of the repair, and a full warm start adds the rest.

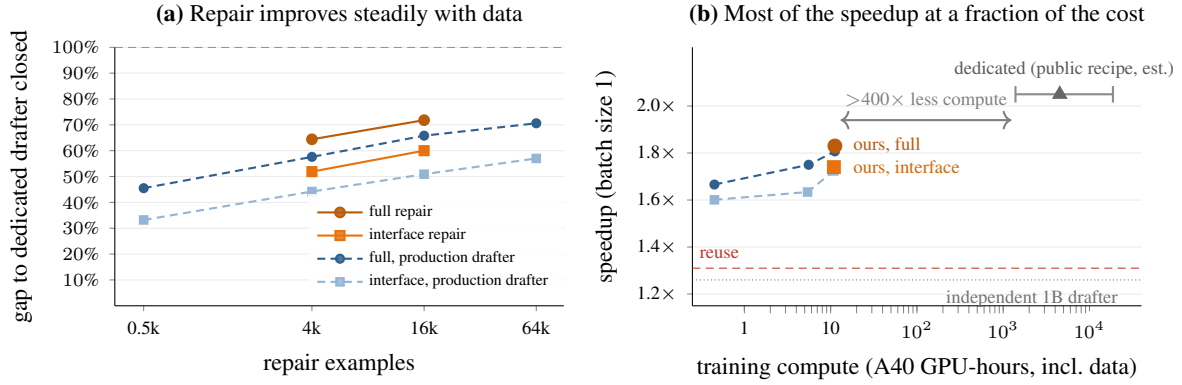


Figure 3: **(a)** Gap to the dedicated drafter closed on R1-Distill-Llama-8B as a function of the number of repair examples, for the EAGLE authors’ family drafter (solid) and the production drafter (dashed). **(b)** Batch-size-1 speedup against training compute, including data generation. Repair attains most of the dedicated drafter’s speedup with orders of magnitude less compute; the dedicated drafter’s range spans 10 to 40 epochs of the public recipe.

7 Related Work

Speculative decoding. Speculative decoding (Leviathan et al., 2023; Chen et al., 2023) has been extended with tree-structured verification (Miao et al., 2024; Chen et al., 2024), self-speculation (Zhang et al., 2024a), and model-free drafting from n-grams, retrieval or suffix structures (Fu et al., 2024; He et al., 2024; Saxena, 2023; Oliaro et al., 2024); see Xia et al. (2024) for a survey. The strongest drafters condition on the target’s hidden states, from Medusa and Hydra heads (Cai et al., 2024; Ankner et al., 2024) to the EAGLE family (Li et al., 2024b,a, 2025), harmonized representations (Zhang et al., 2024b) and block-diffusion drafters (Chen et al., 2026b). Open frameworks such as SpecForge (Li et al., 2026) and speculators (Red Hat AI and vLLM Project, 2025) make training such drafters possible but not cheap, and complementary work reduces their cost by pruning the draft vocabulary (Zhao et al., 2025; Goel et al., 2025). We take these drafters as given and ask how to keep them working when their target changes.

Adapting drafters. DistillSpec (Zhou et al., 2024) distills a draft model from its target, and online speculative decoding (Liu et al., 2024) keeps updating it on served traffic. Other work aligns draft models with chat-tuned targets (Goel et al., 2024), adapts them with parameter- and data-efficient updates (Lin et al., 2026), or evolves them online across vocabularies and targets (Ramakrishnan et al., 2025; Zhang et al., 2026). These approaches treat the drafter as a standalone language model to be aligned with a new target. For target-conditioned drafters, we show that the mis-

alignment caused by post-training is concentrated in a single component, the feature interface. Re-fitting it offline from public instructions recovers most of a dedicated drafter’s speedup, without traffic logs and without original training data.

Post-training and representation drift. Fine-tuning distorts pretrained features (Kumar et al., 2022), and the extent depends on the training signal: on-policy RL forgets less than supervised training on fixed data (Shenfeld et al., 2026; Chen et al., 2026a). The open-model ecosystem consists largely of such derivatives (Horwitz et al., 2025), and reasoning distills (DeepSeek-AI, 2025; Bercovich et al., 2025) and multi-stage post-training pipelines (Lambert et al., 2024) are among its most widely deployed members. Our census connects these findings to inference efficiency. The same regimes that move a model furthest from its initialization are the ones that break the drafters built for it.

8 Conclusion

Family drafters make speculative decoding practical across the open-model ecosystem, but they silently lose much of their value on derivatives distilled from stronger reasoning teachers. We showed that this loss is concentrated in the drafter’s feature interface and that re-fitting this interface on the derivative’s own responses to public instructions restores most of a dedicated drafter’s speedup for a few GPU-hours. Interface repair turns per-derivative drafting from an expensive training project into a routine step that can accompany every model release. More broadly, our re-

sults suggest that the coupling that makes target-conditioned drafters fast need not make them brittle: when the target moves, it suffices to teach the drafter how to read it again.

Limitations

Our study focuses on 8B-scale derivatives of the Llama-3.1 and Qwen3 families and on the EAGLE-3 and DFlash drafters, which are among the most widely deployed configurations. Repair requires running the derivative to generate responses; this cost is small and is included in all our compute figures. We measure wall-clock speedups with greedy decoding on NVIDIA A40 GPUs in vLLM; absolute speedups vary with hardware, batch size and decoding configuration.

References

Talor Abramovich, Maor Ashkenazi, Izzy Puterman, Benjamin Chislett, Tiyasa Mitra, Bitu Darvish Rouhani, Ran Zilberstein, and Yonatan Geifman. 2026. SPEED-Bench: A unified and diverse benchmark for speculative decoding. In *International Conference on Machine Learning (ICML)*.

Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. 2024. Hydra: Sequentially-dependent draft heads for medusa decoding. *arXiv preprint arXiv:2402.05109*.

Akhiad Bercovich et al. 2025. Llama-Nemotron: Efficient reasoning models. *arXiv preprint arXiv:2505.00949*.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.

Howard Chen, Noam Razin, Karthik Narasimhan, and Danqi Chen. 2026a. Retaining by doing: The role of on-policy data in mitigating forgetting. In *International Conference on Machine Learning (ICML)*.

Jian Chen, Yesheng Liang, and Zhijian Liu. 2026b. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*.

Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. 2024. Sequoia: Scalable, robust, and

hardware-aware speculative decoding. *arXiv preprint arXiv:2402.12374*.

DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. Break the sequential dependency of LLM inference using lookahead decoding. In *International Conference on Machine Learning (ICML)*.

Raghav Goel, Mukul Gagrani, Wonseok Jeon, Junyoung Park, Mingu Lee, and Christopher Lott. 2024. Direct alignment of draft model for speculative decoding with chat-fine-tuned LLMs. *arXiv preprint arXiv:2403.00858*.

Raghav Goel et al. 2025. VocabTrim: Vocabulary pruning for efficient speculative decoding in LLMs. *arXiv preprint arXiv:2506.22694*.

Aaron Grattafiori et al. 2024. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Zhenyu He, Zexuan Zhong, Tianle Cai, Jason D. Lee, and Di He. 2024. REST: Retrieval-based speculative decoding. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Elihu Horwitz, Nitzan Kurer, Jonathan Kahana, Liel Amar, and Yedid Hoshen. 2025. Charting and navigating Hugging Face’s model atlas. *arXiv preprint arXiv:2503.10633*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.

Yoon Kim and Alexander M. Rush. 2016. Sequence-level knowledge distillation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Ananya Kumar, Aditi Raghunathan, Robbie Jones, Tengyu Ma, and Percy Liang. 2022. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *International Conference on Learning Representations (ICLR)*.

612	Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying	Gabriele Oliaro, Zhihao Jia, Daniel Campos, and Aurick	665
613	Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gon-	Qiao. 2024. SuffixDecoding: A model-free approach	666
614	zalez, Hao Zhang, and Ion Stoica. 2023. Efficient	to speeding up large language model inference. <i>arXiv</i>	667
615	memory management for large language model serv-	<i>preprint arXiv:2411.04975</i> .	668
616	ing with PagedAttention. In <i>ACM Symposium on</i>		
617	<i>Operating Systems Principles (SOSP)</i> .		
618	Nathan Lambert, Jacob Morrison, Valentina Pyatkin,	Ramchalam Kinattinkara Ramakrishnan, Zhaocong	669
619	Shengyi Huang, Hamish Ivison, Faeze Brahman,	Yuan, Shaojie Zhuo, Chen Feng, Yicheng Lin, Chen-	670
620	Lester James V. Miranda, Alisa Liu, Nouha Dziri,	zheng Su, and Xiaopeng Zhang. 2025. OmniDraft:	671
621	Shane Lyu, et al. 2024. Tulu 3: Pushing frontiers in	A cross-vocabulary, online adaptive drafter for on-	672
622	open language model post-training. <i>arXiv preprint</i>	device speculative decoding. In <i>Advances in Neural</i>	673
623	<i>arXiv:2411.15124</i> .	<i>Information Processing Systems (NeurIPS)</i> .	674
624	Yaniv Leviathan, Matan Kalman, and Yossi Matias.	Red Hat AI and vLLM Project. 2025. Speculators:	675
625	2023. Fast inference from transformers via spec-	A unified library for speculative decoding algo-	676
626	ulative decoding. In <i>International Conference on</i>	rithms in LLM serving. https://github.com/	677
627	<i>Machine Learning (ICML)</i> .	vllm-project/speculators .	678
628	Shenggui Li, Chao Wang, Yikai Zhu, Yubo Wang, Fan	Apoorv Saxena. 2023. Prompt lookup decod-	679
629	Yin, Shuai Shi, et al. 2026. SpecForge: A flexible and	ing. https://github.com/apoorvumang/	680
630	efficient open-source training framework for specula-	prompt-lookup-decoding .	681
631	tive decoding. <i>arXiv preprint arXiv:2603.18567</i> .		
632	Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang	Idan Shenfeld, Jyothish Pari, and Pulkit Agrawal. 2026.	682
633	Zhang. 2024a. EAGLE-2: Faster inference of lan-	RL’s razor: Why online reinforcement learning for-	683
634	guage models with dynamic draft trees. In <i>Proceed-</i>	gets less. In <i>International Conference on Learning</i>	684
635	<i>ings of the Conference on Empirical Methods in Nat-</i>	<i>Representations (ICLR)</i> .	685
636	<i>ural Language Processing (EMNLP)</i> .		
637	Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang	Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann	686
638	Zhang. 2024b. EAGLE: Speculative sampling re-	Dubois, Xuechen Li, Carlos Guestrin, Percy Liang,	687
639	quires rethinking feature uncertainty. In <i>Internat-</i>	and Tatsunori B. Hashimoto. 2023. Stanford Alpaca:	688
640	<i>ional Conference on Machine Learning (ICML)</i> .	An instruction-following LLaMA model. https://	689
641	Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang	github.com/tatsu-lab/stanford_alpaca .	690
642	Zhang. 2025. EAGLE-3: Scaling up inference ac-	Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang,	691
643	celeration of large language models via training-time	Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhi-	692
644	test. <i>arXiv preprint arXiv:2503.01840</i> .	fang Sui. 2024. Unlocking efficiency in large lan-	693
645	Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri	guage model inference: A comprehensive survey of	694
646	Edwards, Bowen Baker, Teddy Lee, Jan Leike, John	speculative decoding. In <i>Findings of the Association</i>	695
647	Schulman, Ilya Sutskever, and Karl Cobbe. 2024.	<i>for Computational Linguistics: ACL 2024</i> .	696
648	Let’s verify step by step. In <i>International Conference</i>		
649	<i>on Learning Representations (ICLR)</i> .	An Yang et al. 2025. Qwen3 technical report. <i>arXiv</i>	697
650	Luxi Lin, Zhihang Lin, Zhanpeng Zeng, Yuhao Chen,	<i>preprint arXiv:2505.09388</i> .	698
651	Qingyu Zhang, Jixiang Luo, Xuelong Li, and Ron-		
652	grong Ji. 2026. Efficiently aligning draft models	Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen,	699
653	via parameter- and data-efficient adaptation. <i>arXiv</i>	Gang Chen, and Sharad Mehrotra. 2024a. Draft &	700
654	<i>preprint arXiv:2603.09527</i> .	verify: Lossless large language model acceleration	701
655	Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Ion Stoica,	via self-speculative decoding. In <i>Proceedings of the</i>	702
656	Zhijie Deng, Alvin Cheung, and Hao Zhang. 2024.	<i>Annual Meeting of the Association for Computational</i>	703
657	Online speculative decoding. In <i>International Con-</i>	<i>Linguistics (ACL)</i> .	704
658	<i>ference on Machine Learning (ICML)</i> .	Lefan Zhang, Xiaodan Wang, Yanhua Huang, and Rui-	705
659	Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao	wen Xu. 2024b. Learning harmonized represen-	706
660	Cheng, et al. 2024. SpecInfer: Accelerating large	tations for speculative sampling. <i>arXiv preprint</i>	707
661	language model serving with tree-based speculative	<i>arXiv:2408.15766</i> .	708
662	inference and verification. In <i>ACM International</i>	Shuyu Zhang, Lingfeng Pan, Qicheng Wang, Yaqi Shi,	709
663	<i>Conference on Architectural Support for Program-</i>	Yueyang Tan, Ruyi Yan, Jiaqi Chen, Lixing Du, and	710
664	<i>ming Languages and Operating Systems (ASPLOS)</i> .	Lu Wang. 2026. EvoSpec: Evolving speculative de-	711
		coding via real-time vocabulary and parameter adap-	712
		tation. <i>arXiv preprint arXiv:2605.27390</i> .	713
		Weilin Zhao et al. 2025. FR-Spec: Accelerating	714
		large-vocabulary language models via frequency-	715
		ranked speculative sampling. <i>arXiv preprint</i>	716
		<i>arXiv:2502.14856</i> .	717

Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. 2024. DistillSpec: Improving speculative decoding via knowledge distillation. In *International Conference on Learning Representations (ICLR)*.

The two reasoning distills are evaluated on SPEED-Bench with 128 prompts.

A Implementation Details

Repair. We optimize Eq. (2) with AdamW (learning rate 2×10^{-5} , weight decay 0.01, cosine schedule with 3% warm-up) for one epoch over the 16k repair examples, packing roughly 2k response tokens per step, with $k_{\max}=3$ training-time-test steps. The loss is computed on response positions only. Repair data are generated greedily with vLLM, capped at 512 response tokens, and deduplicated exactly and approximately against all evaluation prompts. The interface variant updates $W_{fc} \in \mathbb{R}^{4096 \times 12288}$; the full variant updates the interface, the decoder layer and the language-modeling head, keeping the token embedding shared with the target.

Component analysis. For Table 1, all variants use the same 256 R1-Distill responses and 300 optimization steps. LoRA adapters use rank 16 on the query, value and MLP projections of the decoder layer. The feature-statistics baseline fits a per-layer affine map that matches the mean and scale of the derivative’s tapped features to the parent’s.

Evaluation. All acceptance measurements use vLLM with greedy decoding, $K=4$ draft tokens (DFlash: its native block of ten), a maximum of 512 new tokens and batch size 8. Prompts that produce no speculative step are excluded pairwise across compared methods. Speedups are measured on 32 SPEED-Bench prompts at batch size 1 and 128 at batch size 8, each in three fresh server processes with three warm repetitions, relative to autoregressive decoding of the same prompts. The dedicated drafter’s training cost is estimated from our measured A40 training throughput for the public EAGLE-3 recipe at 10 to 40 epochs.

B Census Details

The census covers 174 derivatives of Llama-3.1-8B-Instruct and Qwen3-8B that load with the parent’s architecture and tokenizer. Post-training histories are taken from model cards and grouped as in Figure 1a. Retention is computed per derivative on 64 prompts from its own domain and on 128 SPEED-Bench prompts, and aggregated within each group.